AI335L Deep Learning Lab

Experiment No. 12 | Phase 5

Code Documentation Report

Low-Light Image Enhancement and Object Detection

Zero-DCE + YOLOv8 Pipeline

Course	AI335L Deep Learning
Phase	Phase 5
Team Members	Nimra Waseem S2024AI002 Umair Naveed S2024AI007 Dayan Amjad S2024AI007
Repository	https://github.com/Nimra-waseemaftab/image_enhancement_and_detection
Commit Hash	bc5b0531e7b720af90b97b220cabd291960bb465
Submission Tag	Phase5submission
Date	June 1, 2026

Table of Contents

1. Project Overview and Pipeline Summary

This notebook implements a complete deep learning pipeline for low-light image enhancement followed by object detection. The system combines two state-of-the-art models: Zero-DCE (Zero-Reference Deep Curve Estimation) for unsupervised brightness correction, and YOLOv8 for downstream object detection on the enhanced output.

1.1 Pipeline Architecture

The pipeline consists of five sequential phases, each building upon the previous:

Phase	Label	Description
Phase 1	SETUP	Environment configuration, library imports, GPU detection, seed management, and centralised hyperparameter dictionary.
Phase 2	EDA	Dataset loading, brightness distribution analysis, channel statistics, resolution profiling, outlier detection, and train/test comparison.
Phase 3	MODEL	DCENet architecture, DCENet-Pro variant (with BatchNorm and Dropout), MLP baseline, four physics-inspired unsupervised loss functions, and dataset classes.
Phase 4	TRAIN	Full training infrastructure with AMP, gradient clipping, CosineAnnealing scheduler, early stopping, checkpoint resume, ablation runs, transfer learning, VAE generative component, and Optuna HPO.
Phase 5	INFERENCE	Enhancement inference, YOLOv8 object detection, evaluation metrics (PSNR, SSIM), and GUI deployment.

1.2 Dataset

The LOL-v2 (Low-Light v2) dataset is used throughout. It contains two distinct subsets:

- Synthetic subset: approximately 689 paired train images, synthetically degraded from normal-light originals.
- Real-Captured subset: approximately 100 paired train images captured under genuine dark conditions.

All training is performed on the combined set of both subsets. Normal-light counterparts are retained solely for PSNR/SSIM evaluation. The test split is kept strictly isolated until the final Phase 5 evaluation ceremony.

1.3 Core Metrics

Metric	Description	Higher is Better
PSNR (dB)	Peak Signal-to-Noise Ratio — measures pixel-level fidelity against reference.	Yes
SSIM	Structural Similarity Index — measures perceptual similarity (luminance, contrast, structure).	Yes
MSE	Mean Squared Error — used in MLP baseline and VAE reconstruction tracking.	No
Val Loss	Composite unsupervised loss (Zero-DCE) used for model selection during training.	No

2. Phase 1: Environment Setup

2.1 Core Imports and Device Detection

The first cell imports all required libraries and detects the available compute device. PyTorch and torchvision provide the deep learning foundation; PIL handles image I/O; NumPy and matplotlib support numerical computation and visualisation.

```
import os, glob, time, random
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

2.2 Deterministic Seed Management

Reproducibility is ensured by fixing all random sources before any data splitting, model initialisation, or training. The set_seed() function must be called once at program start and before each experiment to guarantee consistent results across runs.

```
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark     = False
    os.environ["PYTHONHASHSEED"]       = str(seed)
```

2.3 Centralised CONFIG Dictionary

All hyperparameters are stored in a single CONFIG dictionary. This design ensures that every module reads from one authoritative source, eliminating silent inconsistencies between training, validation, and evaluation code.

Parameter	Value	Purpose
image_size	256	Spatial resolution for all training images.
batch_size	16	Images processed per gradient update step.
val_split	0.1	Fraction of training data reserved for validation.
lr	1e-4	Initial Adam learning rate.
weight_decay	1e-4	L2 regularisation coefficient.
grad_clip	1.0	Maximum gradient norm before clipping.

scheduler_step	50	StepLR: reduce LR every N epochs.
scheduler_gamma	0.5	StepLR: LR multiplier at each step.
epochs	200	Maximum training epochs.
early_stop_patience	20	Epochs without improvement before stopping.
use_amp	True	Enable Automatic Mixed Precision on CUDA.
w_smooth	200	Weight for Illumination Smoothness Loss.
w_spatial	1	Weight for Spatial Consistency Loss.
w_color	5	Weight for Colour Constancy Loss.
w_exposure	10	Weight for Exposure Loss.

2.4 Dataset Path Configuration

Dataset paths are declared as constants and grouped into combined lists for training and testing. The notebook uses glob patterns to collect all PNG and JPG files from each directory, supporting both the Synthetic and Real-Captured subsets of LOL-v2.

```
SYNTH_TRAIN_LOW    = r"E:\LOL-v2\LOL-v2\Synthetic\Train\Low"
SYNTH_TRAIN_NORMAL = r"E:\LOL-v2\LOL-v2\Synthetic\Train\Normal"
# ... (all 8 path constants defined similarly)

TRAIN_DIRS = [SYNTH_TRAIN_LOW, REAL_TRAIN_LOW]
TRAIN_FILES_ALL = []
for d in TRAIN_DIRS:
    TRAIN_FILES_ALL += glob.glob(os.path.join(d, "*.png")) \
                     + glob.glob(os.path.join(d, "*.jpg"))
```

3. Phase 2: Exploratory Data Analysis (EDA)

3.1 File Integrity Check

Before any training, every file in the dataset is validated. The check_files() function verifies file existence, non-zero file size, and image decodability using PIL. Corrupted files are excluded and logged. The cleaned file lists replace the originals for all downstream use.

```
def check_files(file_list, label=""):
    good, bad = [], []
    for f in file_list:
        if not os.path.exists(f):
            bad.append((f, "File not found")); continue
        if os.path.getsize(f) == 0:
            bad.append((f, "Zero-byte file")); continue
        try:
            with Image.open(f) as img:
                img.verify()
```

```
        except Exception as e:
            bad.append((f, str(e))); continue
        good.append(f)
    return good, bad
```

3.2 Brightness Distribution Analysis

Mean pixel intensity is computed for a random sample of training images and displayed as a histogram. This analysis directly motivates the Exposure Loss target value. If the dataset average brightness is in the 40–80/255 range, targeting 0.6 (approximately 153/255) in the loss is appropriate.

Key finding from EDA: average brightness across both LOL-v2 subsets falls in the 40–80/255 range, confirming that aggressive enhancement is necessary.

3.3 RGB Channel Distribution

Per-channel pixel intensity histograms are computed across a sample of training images. This analysis reveals colour imbalance caused by sensor performance differences under low-light conditions. The typical pattern in indoor environments is that the red channel has a higher mean than the blue channel, directly motivating the Colour Constancy Loss in Zero-DCE.

3.4 Resolution and Aspect Ratio Analysis

Width, height, and aspect ratio are collected from a sample of training images and plotted. The LOL-v2 dataset contains images with variable resolutions ranging from approximately 400 to 960 pixels wide. Aspect ratios cluster around 3:2 and 4:3. Since all images are resized to 256 x 256 for training, minor distortion occurs but is acceptable for the unsupervised enhancement task.

3.5 Outlier Detection using IQR

Brightness outliers are identified using the interquartile range method. Images with brightness below Q1 - 1.5*IQR (extremely dark) or above Q3 + 1.5*IQR (accidentally normal-light) are flagged. These outliers could destabilise unsupervised training because the loss functions assume the input is genuinely dark.

```
Q1, Q3 = np.percentile(all_brightness, 25), np.percentile(all_brightness, 75)
IQR    = Q3 - Q1
lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
outliers_dark   = np.where(all_brightness < lo)[0]
outliers_bright = np.where(all_brightness > hi)[0]
```

3.6 Train vs Test and Synthetic vs Real Distribution Comparison

Two additional EDA cells compare brightness distributions across dataset splits. The first verifies that train and test sets have similar brightness profiles (mean difference below 20 intensity units is acceptable). The second

quantifies the domain gap between the Synthetic and Real-Captured subsets, which typically differ by 10–20 intensity units. Understanding this gap sets expectations for generalisation performance.

3.7 EDA Findings Summary

Finding	Value / Observation	Implication
Average brightness	40–80 out of 255	Exposure loss target of 0.6 is appropriate.
RGB channel imbalance	R mean > G mean > B mean	Colour Constancy Loss is essential.
Image resolution range	400–960 px wide	Resize to 256x256 is acceptable.
Corrupted files	Zero detected	All files used without exclusion.
Synthetic vs Real gap	10–20 intensity units	Model must generalise across both splits.
Train vs test distribution	Mean diff < 20 units	No significant distribution shift detected.

4. Phase 3: Model Architecture and Loss Functions

4.1 Data Pipeline: LowLightDataset and Augmentation

Two dataset classes are defined. The base LowLightDataset accepts either a list of file paths or a single directory and applies a deterministic validation transform. The augmented variant applies a five-stage augmentation pipeline to training images only, preventing augmented versions from appearing in validation.

Augmentation	Parameters	Purpose
RandomHorizontalFlip	p=0.5	Doubles effective dataset size without adding images.
RandomRotation	degrees=15	Adds orientation variance to prevent directional bias.
ColorJitter	brightness=0.3, contrast=0.3	Simulates lighting variation across captures.
RandomResizedCrop	scale=(0.8,1.0)	Provides scale invariance and additional spatial diversity.
ToTensor	N/A	Converts [0,255] uint8 PIL images to [0,1] float32 tensors.

The train/validation split is performed before dataset construction to prevent data leakage. Shuffling is done with a fixed seed before slicing so the split is identical on every run.

4.2 Original DCENet Architecture

DCENet is a seven-layer convolutional network with U-Net-style skip connections. The encoder builds increasingly abstract feature representations across four convolutional layers. The decoder reconstructs spatial resolution by concatenating encoder features (skip connections) before each decoding layer. The final layer outputs 24 channels representing eight iterations of three per-channel curve parameter maps.

```
class DCENet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3,  32, 3, padding=1)   # Encoder
        self.conv2 = nn.Conv2d(32, 32, 3, padding=1)
        self.conv3 = nn.Conv2d(32, 32, 3, padding=1)
        self.conv4 = nn.Conv2d(32, 32, 3, padding=1)
        self.conv5 = nn.Conv2d(64, 32, 3, padding=1)   # Decoder (skip)
        self.conv6 = nn.Conv2d(64, 32, 3, padding=1)
        self.conv7 = nn.Conv2d(64, 32, 3, padding=1)
        self.final = nn.Conv2d(32, 24, 3, padding=1)   # 24 = 8 iters x 3 ch

    def forward(self, x):
        x1 = self.relu(self.conv1(x))
        x2 = self.relu(self.conv2(x1))
        x3 = self.relu(self.conv3(x2))
        x4 = self.relu(self.conv4(x3))
        x5 = self.relu(self.conv5(torch.cat([x4, x3], 1)))
        x6 = self.relu(self.conv6(torch.cat([x5, x2], 1)))
        x7 = self.relu(self.conv7(torch.cat([x6, x1], 1)))
        return self.tanh(self.final(x7))
```

Total parameters: approximately 79,000. The model is intentionally compact to avoid overfitting on the relatively small LOL-v2 training set (approximately 800 images combined).

4.3 DCENet-Pro: Enhanced Architecture

DCENet-Pro extends the original with three architectural enhancements required by the project specification: BatchNorm2d after every convolutional layer, MaxPool2d downsampling in the encoder, and Spatial Dropout2d on the final feature map. The decoder uses bilinear upsampling to recover spatial resolution.

Enhancement	Component	Benefit
Batch Normalisation	nn.BatchNorm2d(out_ch)	Reduces internal covariate shift, stabilises training.
MaxPooling	nn.MaxPool2d(2, 2)	Provides translation invariance, reduces spatial resolution.
Spatial Dropout	nn.Dropout2d(p=0.1)	Regularises spatial feature maps, reduces co-adaptation.
Bilinear Upsample	nn.Upsample(scale_factor=2)	Smooth spatial upsampling without checkerboard artefacts.
Kaiming Init	_init_weights()	Appropriate initialisation for ReLU activations.

4.4 MLP Baseline

An MLP baseline processes each pixel independently as a 3-dimensional (R, G, B) vector. The network has two hidden layers with 64 units each, BatchNorm, ReLU activations, and Dropout. Because it processes pixels independently, the MLP has no spatial awareness and cannot leverage local texture or structure information. This establishes a lower-bound performance reference for evaluating the convolutional models.

```
class MLPBaseline(nn.Module):
    def __init__(self, hidden=64, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(3, hidden), nn.BatchNorm1d(hidden),
            nn.ReLU(), nn.Dropout(p=dropout),
            nn.Linear(hidden, hidden), nn.BatchNorm1d(hidden),
            nn.ReLU(), nn.Dropout(p=dropout),
            nn.Linear(hidden, 3), nn.Sigmoid()
        )

    def forward(self, x):
        B, C, H, W = x.shape
        flat = x.permute(0, 2, 3, 1).reshape(-1, 3)
        out  = self.net(flat)
        return out.reshape(B, H, W, 3).permute(0, 3, 1, 2)
```

4.5 Loss Functions

Zero-DCE is trained without reference images. Four physics-inspired loss functions define what constitutes good enhancement:

Loss Function	Formula (Conceptual)	Prevents	Weight
Illumination Smoothness	Sum of |r[x] - r[x+1]| across spatial dims	Patchy or noisy curve maps	200
Spatial Consistency	Sum of (Laplacian(input) - Laplacian(output))^2	Destruction of local contrast	1
Colour Constancy	sqrt((R-G)^2 + (R-B)^2 + (G-B)^2) on means	Colour shift and tinting	5
Exposure	Sum of (AvgPool(output) - 0.6)^2	Under- or over-exposure	10

4.6 Enhancement Curve

Rather than predicting an enhanced image directly, DCENet predicts curve parameters. The curve formula x_new = x + r*(x^2 - x) is applied iteratively eight times. For positive r the curve is concave-up and brightens the image. For negative r the curve darkens it. Applying the formula eight times allows compound adjustments from a single model forward pass.

```
def enhance(x, r):
    for i in range(0, 24, 3):
        r_i = r[:, i:i+3, :, :]
        x = x + r_i * (x * x - x)
    return x
```

5. Phase 4 Core: Training Loop and Ablation Study

5.1 Full Training Infrastructure: train_zero_dce()

The modular training function encapsulates the complete training infrastructure. It accepts a model class, model path, data loaders, and a config dictionary, making it reusable across all ablation configurations.

Feature	Implementation	Rationale
Optimiser	Adam with weight_decay=1e-4	L2 regularisation prevents overfitting.
LR Schedule	CosineAnnealingLR (T_max=epochs)	Smoothly reduces LR; avoids abrupt step drops.
Gradient Clipping	clip_grad_norm_(model, grad_clip=1.0)	Prevents exploding gradients in early training.
Mixed Precision	GradScaler + autocast (CUDA only)	Reduces memory usage, speeds up GPU training.
Checkpoint Resume	Saves model, optimiser, and scheduler	Allows training to continue after interruption.
Early Stopping	Patience=20 on validation loss	Stops training when improvement plateaus.
Best Model Save	torch.save on new val loss minimum	Guarantees best checkpoint is always preserved.

5.2 Experiment Tracking: MetricWriter

A lightweight CSV-based metric logger replaces TensorBoard to avoid NumPy 2.x compatibility issues. The MetricWriter class writes all scalar metrics (train loss, validation loss, learning rate, PSNR, SSIM) to a CSV file and can generate plots without requiring TensorBoard.

```
class MetricWriter:
    def add_scalar(self, tag, value, step):
        self._data.setdefault(tag, []).append((step, float(value)))
        self._writer.writerow([step, tag, float(value)])
        self._file.flush()

    def plot(self, tags=None):
        # plots all logged scalar curves after training
        ...
```

5.3 Interpretability: Feature Maps and Grad-CAM

Two interpretability visualisations are implemented. The first uses a forward hook to extract and display feature map activations at any named convolutional layer. The second implements Vanilla Gradient Saliency (magnitude

of input gradients) and Grad-CAM (gradient-weighted class activation maps) to highlight which spatial regions the network attends to when making enhancement decisions.

```
def grad_cam_zerodce(model, img_pil, target_layer="conv4"):
    activations, gradients = {}, {}
    # Register forward and backward hooks
    fh = target.register_forward_hook(fwd_hook)
    bh = target.register_full_backward_hook(bwd_hook)
    # Forward pass, compute loss, backpropagate
    r = model(x); enhanced = enhance(x, r); loss = enhanced.mean()
    loss.backward()
    # Compute weighted activation map
    weights = gradients["feat"].mean(dim=(1, 2), keepdim=True)
    cam = torch.relu((weights * activations["feat"]).sum(dim=0))
    ...
```

5.4 Ablation Study: Three Regularisation Configurations

Three ablation configurations isolate the contribution of different regularisation components. All configurations use the same training function, data splits, seed, and evaluation metrics to ensure comparability.

Run	Model	Configuration	Weight Decay
Run 1: No Regularisation	DCENetRun1 (DCENet)	No Dropout, no augmentation, no weight decay.	0.0
Run 2: Drop + Aug + WD	DCENetRun2	Dropout2d after final conv, augmented data loader, weight decay.	1e-4
Run 3: BN + Reg (DCENet-Pro)	DCENetPro	Full BatchNorm + Dropout + augmentation + weight decay.	1e-4

Each ablation is trained for 40 epochs. Results are reported as best validation loss, training time, and parameter count. If a checkpoint exists on disk, training is skipped and the saved model is loaded automatically.

5.5 Three-Seed Comparison Table

The COMPARISON dictionary collects per-seed PSNR and SSIM values for the MLP Baseline, original Zero-DCE, and DCENet-Pro across three seeds (42, 123, 7). Mean and standard deviation are reported to quantify variance from non-determinism.

```
SEEDS = [42, 123, 7]
for name, model_fn, mpath in models_to_compare:
    p_list, s_list = [], []
    for seed in SEEDS:
        p, s = quick_eval_model(model_fn, mpath, seed, n_eval=10)
        p_list.append(p); s_list.append(s)
    COMPARISON[name] = {
        "psnr_mean": np.mean(p_list), "psnr_std": np.std(p_list),
        "ssim_mean": np.mean(s_list), "ssim_std": np.std(s_list),
    }
```

6. Phase 4 Workstream 1: Transfer Learning Study

6.1 Source Model and Architecture

Transfer learning uses MobileNetV2 pretrained on ImageNet-1K (torchvision, Apache 2.0 licence) as the encoder backbone. MobileNetV2 was selected for its compact size (~3.4M parameters), strong ImageNet feature representations, and efficient depthwise separable convolutions that work well as a starting point for dense prediction tasks.

The TLModel class wraps the MobileNetV2 feature extractor with a custom lightweight decoder head for image-to-image enhancement:

Component	Architecture	Output Shape
Encoder	MobileNetV2 features (frozen or trainable)	(B, 1280, H/32, W/32)
Pool	AdaptiveAvgPool2d((8, 8))	(B, 1280, 8, 8)
reduce	Conv2d(1280 -> 256, kernel=1)	(B, 256, 8, 8)
up1	Conv2d + ReLU + PixelShuffle(2)	(B, 64, 16, 16)
up2	Conv2d + ReLU + PixelShuffle(2)	(B, 32, 32, 32)
up3	Conv2d + ReLU + PixelShuffle(2)	(B, 16, 64, 64)
up4	Conv2d + ReLU + Upsample(x4) + Conv2d	(B, 3, 256, 256)

6.2 Freeze and Unfreeze API

TLModel exposes three methods for controlling which parameters are trainable at any point in the training process:

```
def freeze_backbone(self):
    for p in self.encoder.parameters():
        p.requires_grad_(False)

def unfreeze_backbone(self, from_block=0):
    # Unfreeze all blocks at or after from_block index
    for p in self.encoder.parameters():
        p.requires_grad_(True)
    if from_block > 0:
        for name, p in self.encoder.named_parameters():
            if int(name.split(".")[0]) < from_block:
                p.requires_grad_(False)

def param_counts(self):
    total     = sum(p.numel() for p in self.parameters())
    trainable = sum(p.numel() for p in self.parameters()
                    if p.requires_grad)
    return total, total - trainable, trainable
```

6.3 Shared Training Loop: train_tl()

All four TL configurations use a single shared training function to guarantee a fair comparison. The function supports differential learning rates via a per-parameter-group list in the optimiser configuration. Evaluation uses PSNR and SSIM computed on batches of validation pairs.

Design Choice	Implementation
Loss function	L1 (MAE) — more robust to outlier pixels than MSE for image restoration.
Optimiser	Adam with weight_decay=1e-4 and cosine LR annealing.
Metrics	PSNR and SSIM computed per batch on validation pairs.
Differential LR	Separate Adam parameter groups for head and backbone parameters.
Checkpoint	Best validation PSNR checkpoint saved to disk after each configuration.
Fast validation	Limited to 10 validation batches per epoch for speed.

6.4 Four Transfer Learning Configurations

Configuration A — Feature Extraction: The backbone is fully frozen. Only the decoder head parameters are trained. This is the safest approach as it preserves all ImageNet features. It trains fastest but may underperform if ImageNet features are misaligned with the low-light domain.

Configuration B — Full Fine-Tuning: All parameters are unfrozen and trained at a single uniform learning rate of 1e-4. This is the highest-capacity approach but carries the greatest risk of catastrophic forgetting — the overwriting of useful low-level ImageNet features by the task-specific gradients.

Configuration C — Differential Learning Rates: The backbone is trained at 1e-5 (100 times smaller than the head rate of 1e-3). This preserves low-level features in the backbone while allowing the head to adapt rapidly to the enhancement task. This strategy typically yields the best balance.

Configuration D — Gradual Unfreezing: A three-phase schedule is used. In Phase 1 (epochs 1-10), the backbone is frozen and only the head trains. In Phase 2 (epochs 11-15), the top quarter of MobileNetV2 blocks (index 12 onward) is unfrozen. In Phase 3 (epochs 16-20), the entire backbone is unfrozen at a small backbone learning rate of 1e-5. This approach prevents early gradient noise from corrupting backbone features.

```
PHASE_SCHEDULE = [
    (range(1,  11), 0,  {"head": 1e-3, "bb": None}),   # head only
    (range(11, 16), 12, {"head": 1e-3, "bb": 1e-4}),   # unfreeze top
    (range(16, 21), 0,  {"head": 1e-3, "bb": 1e-5}),   # full, small LR
]
```

6.5 Paired Dataset for Supervised Transfer Learning

Unlike the unsupervised Zero-DCE training, transfer learning uses paired (low-light, normal-light) images for supervised L1 loss computation. The PairedLowLightDataset class matches each low-light file to its corresponding normal-light reference by filename.

```
def pair_files(low_files, norm_dirs):
    pairs = []
    for lf in low_files:
```

```
        fname = os.path.basename(lf)
        for nd in norm_dirs:
            nf = os.path.join(nd, fname)
            if os.path.exists(nf):
                pairs.append((lf, nf)); break
    return pairs
```

7. Phase 4 Workstream 2: Variational Autoencoder (VAE)

7.1 Purpose and Integration

The VAE generative component addresses the class imbalance between the LOL-v2 subsets: the Synthetic split contains approximately 689 training pairs while the Real-Captured split contains only approximately 100. By training a VAE on all low-light images and sampling from the learned latent distribution, synthetic augmentation data is generated for the underrepresented real-captured domain.

7.2 ConvVAE Architecture

ConvVAE uses a convolutional encoder-decoder pair with a 128-dimensional latent space. The encoder reduces a 3 x 128 x 128 input to a 128-dimensional Gaussian distribution parameterised by mu and log-variance vectors. The reparameterisation trick enables gradients to flow through the sampling operation.

Component	Layer Sequence	Output Shape
Encoder	Conv(3->32,s=2), Conv(32->64,s=2), Conv(64->128,s=2), Conv(128->256,s=2), Flatten	(B, 16384)
FC mu	Linear(16384 -> 128)	(B, 128)
FC logvar	Linear(16384 -> 128)	(B, 128)
Reparameterise	z = mu + eps * exp(0.5 * logvar), eps ~ N(0,1)	(B, 128)
FC decode	Linear(128 -> 16384), reshape	(B, 256, 8, 8)
Decoder	ConvT(256->128,s=2), ConvT(128->64,s=2), ConvT(64->32,s=2), ConvT(32->3,s=2), Sigmoid	(B, 3, 128, 128)

7.3 Reparameterisation Trick

The reparameterisation trick is the key innovation that makes VAE training possible. Direct sampling from the Gaussian posterior N(mu, sigma^2) is not differentiable with respect to mu and sigma. The trick reformulates sampling as z = mu + epsilon * sigma where epsilon ~ N(0, I) is drawn outside the computation graph. Gradients can then flow through mu and sigma during backpropagation.

```
def reparameterise(self, mu, logvar):
    if self.training:
        std = torch.exp(0.5 * logvar)
```

```
        eps = torch.randn_like(std)    # sample epsilon outside graph
        return mu + eps * std          # differentiable operation
    return mu                          # deterministic at eval time
```

7.4 VAE Loss: ELBO with KL Annealing

The Evidence Lower Bound (ELBO) objective combines a reconstruction term and a regularisation term:

Total Loss = MSE_reconstruction + beta * KL_divergence

The KL divergence term KL = -0.5 * sum(1 + logvar - mu^2 - exp(logvar)) measures how far the posterior distribution is from the standard normal prior. Without the KL term the model would behave like a standard autoencoder. Without the reconstruction term the model would collapse to the prior.

KL annealing gradually increases beta from 0 to 1 over the first 20 of 40 training epochs. Starting with beta = 0 forces the decoder to first learn reconstruction, then progressively introduces the latent regularisation. This prevents posterior collapse, where the KL term reaches near-zero and the decoder ignores the latent code.

```
for epoch in range(1, VAE_EPOCHS + 1):
    beta = min(BETA_MAX, BETA_MAX * epoch / ANNEAL_EPOCHS)
    recon_loss = F.mse_loss(recon, imgs, reduction="sum") / batch_size
    kl_loss    = -0.5 * torch.sum(
                     1 + logvar - mu.pow(2) - logvar.exp()
                 ) / batch_size
    loss = recon_loss + beta * kl_loss
```

7.5 Latent Space Analysis

Three latent space analyses are implemented. First, PCA reduces the 128-dimensional mu vectors of up to 300 training images to two dimensions, coloured by Synthetic vs Real-Captured subset membership. Overlap in PCA space indicates that the VAE has learned a shared representation; separation indicates a domain gap that limits augmentation effectiveness.

Second, latent interpolation linearly blends mu_A and mu_B between two real training images at eight equally spaced alpha values. Smooth, perceptually continuous transitions confirm a well-structured latent space.

Third, unconditional sampling (z ~ N(0, I)) generates novel low-light images by decoding random standard-normal latent vectors, producing the 4x4 grid of 16 generated samples for the report.

8. Phase 4 Workstream 3: Hyperparameter Optimisation (HPO)

8.1 Optuna Framework

Hyperparameter search uses Optuna with the Tree-structured Parzen Estimator (TPE) sampler. TPE is a Bayesian optimisation algorithm that models the prior distribution of good and bad hyperparameter configurations and proposes new trials by sampling from regions likely to produce good outcomes.

Persistence is handled by SQLite storage. All trial results are written to a database file (optuna_study.db), allowing the study to resume if interrupted and enabling post-hoc analysis of all trials.

8.2 Search Space Definition

Hyperparameter	Search Range	Distribution
Learning rate	1e-5 to 1e-2	Log-uniform
Weight decay	1e-6 to 1e-2	Log-uniform
Dropout probability	0.0 to 0.4	Uniform float
Smoothness loss weight	50 to 500	Integer uniform
Exposure loss weight	1 to 30	Integer uniform
Colour loss weight	1 to 20	Integer uniform

8.3 Objective Function

Each Optuna trial instantiates a DCENetPro with the sampled dropout probability, trains it for a reduced number of epochs (the fast probe budget), and returns the best validation loss achieved. Pruning via Optuna's MedianPruner terminates unpromising trials early, reducing total compute.

```
def objective(trial):
    lr      = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
    wd      = trial.suggest_float("weight_decay", 1e-6, 1e-2, log=True)
    drop    = trial.suggest_float("dropout", 0.0, 0.4)
    w_s     = trial.suggest_int("w_smooth", 50, 500)
    ...
    model   = DCENetPro(dropout_p=drop).to(device)
    # Train for probe epochs and return best val loss
    return best_val_loss
```

9. Phase 5 Evaluation Plan: Tasks 1–7

Phase 5 shifts the focus from building to understanding. The grade is awarded entirely for the rigour and honesty of the evaluation, not for the absolute test-set number. A team reporting a modest result with deep analysis scores higher than one reporting a strong number with shallow analysis.

9.1 Task 1: Test-Set Ceremony

The test set is evaluated exactly once after freezing the final model configuration from Phase 4. The procedure requires committing and tagging the frozen configuration in git before running evaluation. The evaluation is then run across three random seeds to capture variance from non-determinism (data shuffling, dropout at inference). After this single evaluation, the test set is closed permanently.

- Freeze final model configuration with git commit message "freeze: final config for Phase 5 test evaluation".
- Run test evaluation across three seeds (42, 123, 7) and record all per-seed metrics.
- Tag commit: git tag final-test-evaluation && git push --tags.
- Report mean and standard deviation of PSNR and SSIM across seeds.
- Per-class breakdown (if applicable) and comparison rows for MLP baseline, Phase 3 model, Phase 4 model, and final model.

9.2 Task 2: Ablation Studies

At least five ablations spanning at least three categories are required. The three existing ablation runs (No Regularisation, Dropout + Augmentation + WD, and DCENet-Pro with BN) already cover the "remove/replace a component" category. Additional ablations should span at least two further categories.

Category	Planned Ablation	Status
Remove component	Run 1 (No Regularisation) vs Run 3 (Full Reg)	Implemented
Replace component	Run 2 (Dropout only) vs Run 3 (BN + Dropout)	Implemented
Architecture	DCENet-Pro vs original DCENet	Implemented
Data ablation	Train on 25%, 50%, 75%, 100% of training data	Required
Training recipe	Without LR scheduling vs with CosineAnnealing	Required

9.3 Task 3: Error Analysis

Error analysis examines where and how the model fails. The quantitative component includes pixel-level residual plots, calibration analysis (predicted confidence vs actual accuracy), and per-subgroup performance breakdown (Synthetic vs Real-Captured subsets). The qualitative component requires manually inspecting 30 to 50 worst-performing examples and clustering them into 4 to 6 failure categories.

Hard-example mining identifies the 20 examples with the largest residuals or lowest PSNR. These are inspected to determine whether they are genuinely hard, mislabelled, or out-of-distribution.

9.4 Task 4: Robustness Analysis

For image modality, at least two robustness probes are required. Each probe applies a perturbation of increasing strength and records how PSNR/SSIM degrades. Probes appropriate for this project include:

- Gaussian noise at sigma = 0.01, 0.05, 0.1, 0.2, 0.5.
- JPEG compression at quality = 90, 70, 50, 30, 10.
- Brightness and contrast shifts.
- FGSM adversarial perturbation at epsilon = 0.01, 0.05, 0.1.

For each probe, a degradation curve (metric vs perturbation strength) is plotted. A model that degrades gracefully is more informative than one with a single reported perturbed accuracy.

9.5 Task 5: Efficiency and Compute Reporting

Metric	Tool / Method
Parameter count (total and trainable)	sum(p.numel() for p in model.parameters())
FLOPs per forward pass	ptflops or thop library
Inference latency (single and batched)	torch.cuda.synchronize() + time.time() with warmup runs excluded
Peak GPU memory	torch.cuda.max_memory_allocated()
Total training time	Sum of all final-model and HPO run times from training logs
Checkpoint disk size	os.path.getsize() on .pth files

9.6 Tasks 6 and 7: Literature Comparison and Statistical Reporting

Task 6 compares final results against at least three published results on LOL-v2 or a closely related benchmark. Comparison asymmetries (different splits, different evaluation protocols, different compute scales) must be explicitly noted.

Task 7 requires bootstrap confidence intervals (1,000 resamples) on the primary metric, a paired t-test or equivalent between the final model and the strongest baseline, and reporting effect sizes alongside p-values to distinguish statistical significance from practical significance.

10. Configuration and Hyperparameter Reference

10.1 Training Configurations by Model

Model	Epochs	LR	Weight Decay	Scheduler	Notes
MLP Baseline	50	1e-3	1e-4	StepLR(15, 0.5)	Early stop patience 10.
DCENet (Phase 3)	200	1e-4	0	None	Adam, no L2 reg.
DCENet Full (G)	200	1e-4	1e-4	CosineAnnealing	AMP, gradient clip.
DCENet-Pro (Run 3)	40	1e-4	1e-4	CosineAnnealing	Ablation run.
TL Config A	20	1e-3	1e-4	CosineAnnealing	Frozen backbone.
TL Config C	20	Head: 1e-3, BB: 1e-5	1e-4	Cosine	Differential LR.
VAE	40	2e-4	1e-5	CosineAnnealing	KL annealing beta 0->1 over 20 epochs.

10.2 Checkpoint Files

File	Model	Phase
best_mlp_baseline.pth	MLP Baseline (64 hidden)	Phase 3
best_zero_dce_lolv2.pth	Original DCENet	Phase 3 / Phase 4
ablation_run1.pth	DCENetRun1 (No Regularisation)	Phase 4 Ablation
ablation_run2.pth	DCENetRun2 (Drop + Aug + WD)	Phase 4 Ablation
ablation_run3.pth	DCENetPro (BN + Reg)	Phase 4 Ablation
tl_config_A.pth	TLModel Config A	Phase 4 TL
tl_config_B.pth	TLModel Config B	Phase 4 TL
tl_config_C.pth	TLModel Config C	Phase 4 TL
tl_config_D.pth	TLModel Config D (Gradual)	Phase 4 TL
vae_best.pth	ConvVAE (latent_dim=128)	Phase 4 VAE

11. Key Design Decisions and Deep Learning Concepts

11.1 Why Unsupervised Loss Functions?

Zero-DCE requires no paired training images. This is a significant practical advantage because collecting paired (dark, normal-light) images at scale is expensive and imposes strict scene repeatability requirements. The four loss functions encode domain knowledge about what constitutes good enhancement without ever seeing a reference image.

11.2 Why Curve Estimation Instead of Direct Prediction?

Directly predicting the enhanced image pixel-by-pixel is unconstrained and can produce artefacts. By predicting curve parameters instead, the network is constrained to produce smooth, monotonic transformations. The iterative application (eight rounds of the same curve formula) provides compounding effect without increasing model capacity.

11.3 Why CosineAnnealingLR Instead of StepLR?

StepLR applies abrupt LR drops at fixed intervals, which can cause loss spikes. CosineAnnealingLR provides a smooth reduction from the initial LR to a minimum (eta_min = lr * 0.01) following a cosine curve. This avoids destabilising training while still ensuring the LR reaches very small values by the end of training.

11.4 Why Mixed Precision (AMP)?

Automatic Mixed Precision uses float16 for most computations and float32 for numerically sensitive operations. This reduces GPU memory usage by approximately half and speeds up computation on Tensor Core-equipped GPUs. The GradScaler component compensates for float16's limited dynamic range by scaling gradients before backpropagation and unscaling before the optimiser step.

11.5 Why Kaiming Initialisation?

Kaiming (He) initialisation sets initial weights with variance scaled by 2/fan_in for ReLU activations. This prevents the vanishing gradient problem that occurs when weights are initialised too small (gradients shrink through layers) or the exploding gradient problem when they are too large. Both DCENetPro and MLPBaseline use Kaiming uniform initialisation.

11.6 Leakage Prevention in Train/Val Split

The train/validation split is performed on the raw file list before any Dataset object is constructed. Shuffling is done with a fixed seed before slicing. This sequence ensures that no validation images can accidentally appear in the training augmentation pipeline, which would give an optimistic view of generalisation.

11.7 Phase 5 Evaluation Principles

The Phase 5 evaluation follows the principle that a reported number must be obtained through a single, documented, unrepeatable evaluation. Multiple evaluations on the test set inflate apparent performance because the reported number implicitly absorbs information from all prior evaluations. The git ceremony enforces this constraint procedurally.

Reported results must include mean and standard deviation across at least three seeds. A 0.2 dB PSNR improvement that is statistically significant on a large test set is still a 0.2 dB improvement — effect size matters, not just p-values.